IBM Datathon: Team Brief

Real-time, explainable ECG arrhythmia detection | Track: Real-Time AI for Critical Decisions

The decision in one line

We build a system that watches a live ECG, spots abnormal heart rhythms within seconds, raises an alert, and explains *why* it flagged them in plain language. Medical domain, no India-specific requirement.

1. Key facts

Item	Detail
Platform	Hack2Skill (H2S) portal, IBM Datathon, submission form with a Submit button
Track to select	**Real-Time AI for Critical Decisions** (other options: AI Secured; Wildcard)
Problem	Cardiac arrhythmias are caught late and black-box alerts are not trusted
Approach	1D-CNN on ECG windows + Grad-CAM explanations + IBM Granite (watsonx.ai) plain-language reasons + live Streamlit dashboard
Team size / deadline	Team size: _____ Deadline: _____ (fill in from the portal)

2. Why ECG arrhythmia

- **Fits the track:** a dangerous rhythm needs a decision in seconds, so it is genuinely real-time and critical.
- **Data is open and verified:** both datasets download with no approval or credentialing.
- **Great live demo:** a moving ECG waveform with a risk score and an alert is easy for judges to understand.
- **Measurable:** precision, recall, false-alarm rate and latency can all go on a slide.

3. Datasets (both open access, no approval needed)

Dataset	What it is	Use for	Size
PTB-XL physionet.org/content/ptb-xl/1.0.3	21,799 12-lead ECGs, 18,869 patients, 10 s each, 71 labels	Training the model	1.7 GB zip (3.0 GB unzipped)
MIT-BIH Arrhythmia physionet.org/content/mitdb/1.0.0	48 half-hour two-channel recordings, about 110,000 beat labels	Beat-level validation and live replay demo	73.5 MB zip

Download shortcut for PTB-XL: you only need the `records100` folder, `ptbxl_database.csv` and `scp_statements.csv`. Skip `records500` to save space.

4. Quick start code

```
pip install wfdb pandas

# MIT-BIH: one record + beat labels
import wfdb
sig, meta = wfdb.rdsamp('mitdb/100')
ann = wfdb.rdann('mitdb/100', 'atr')

# PTB-XL: labels + one ECG (100 Hz)
import pandas as pd, ast
df = pd.read_csv('ptb-xl/ptbxl_database.csv', index_col='ecg_id')
df['scp_codes'] = df['scp_codes'].apply(ast.literal_eval)
sig, meta = wfdb.rdsamp('ptb-xl/' + df.filename_lr.iloc[0])
```

Standard split for PTB-XL: use the `strat_fold` column. Train on folds 1-8, validate on fold 9, test on fold 10.

5. Task list

#	Task	What to do	Phase	Owner	Done
1	**Set up**	Create shared repo + Colab/Kaggle notebook. Confirm everyone can open the PhysioNet pages. Check IBM Cloud / watsonx.ai access (fallback: template-based explanations).	Phase 1		[ ]
2	**Get data**	Download PTB-XL (records100 + 2 CSVs) and MIT-BIH. Load one record from each with wfdb to confirm it works.	Phase 1		[ ]
3	**Preprocess**	Band-pass filter, normalise, cut into fixed windows. For MIT-BIH, group beat symbols into classes (e.g. normal, supraventricular, ventricular, fusion, other).	Phase 2		[ ]
4	**Train model**	Baseline first (simple CNN or XGBoost on features), then 1D-CNN. Use weighted loss for class imbalance. Use the strat_fold split for PTB-XL.	Phase 2		[ ]
5	**Evaluate**	Report per-class precision, recall, F1, AUROC, false-alarm rate. Do not report plain accuracy (imbalanced data).	Phase 2		[ ]
6	**Explainability**	Grad-CAM heatmap over the waveform. Send the top findings to an IBM Granite model on watsonx.ai to write a one-line clinical-style reason.	Phase 3		[ ]
7	**Streaming demo**	Replay a MIT-BIH record as a live stream (window by window). Measure and display latency per window.	Phase 3		[ ]
8	**Dashboard**	Streamlit app: live waveform, rolling risk score, Normal / Watch / Critical alert, explanation text, alert history log.	Phase 3		[ ]
9	**Form answers**	Paste the answers from section 6 into the portal. Check the character/word counters on the live form before submitting.	Phase 4		[ ]
10	**Pitch + demo video**	5-slide outline: problem, solution, live demo, metrics (with latency), impact + limitations. Rehearse one end-to-end flow.	Phase 4		[ ]

6. Ready-to-paste form answers

Track: **Real-Time AI for Critical Decisions**

Define the problem statement (limit 200 - this draft is 194 characters)

> Cardiac arrhythmias are often caught late, and black-box alerts are not trusted by clinicians. We build a real-time, explainable ECG monitor that flags abnormal rhythms in seconds and shows why.

Describe your proposed solution (less than 200 words - this draft is 146 words)

> We build a streaming ECG monitoring system that detects arrhythmias within seconds. A live ECG stream is filtered, split into beat-level windows and passed to a 1D convolutional neural network trained on PTB-XL (12-lead ECGs) and validated on MIT-BIH beat annotations. Each window gets a risk score and an alert level: Normal, Watch or Critical. Explainability is built in: Grad-CAM highlights the exact part of the waveform that triggered the alert, and an IBM Granite model on watsonx.ai converts it into a plain-language reason, such as irregular beat intervals with wide QRS complexes. A Streamlit dashboard shows the live waveform, rolling risk score, alert history and the explanation. We handle class imbalance with weighted loss and report per-class precision, recall and F1, false-alarm rate and end-to-end latency per window. Predictions are logged for audit. The system is decision support for clinicians, not a replacement for diagnosis.

Audience (numbered bullets) (limit 1024 characters - this draft is 347)

> 1. Cardiologists and ICU nurses - need instant, prioritised rhythm alerts they can verify.
> 2. Small hospitals and clinics with few specialists - get automated first-level ECG screening.
> 3. Ambulance and emergency teams - early warning before the patient reaches hospital.
> 4. Patients using wearable ECG monitors - faster alerts with clear reasons.

Datasets (limit 1024 characters - this draft is 416)

1. PTB-XL - 21,799 clinical 12-lead ECGs from 18,869 patients with 71 diagnostic, form and rhythm labels (PhysioNet, open access). Main training data.
2. MIT-BIH Arrhythmia Database - 48 half-hour two-channel recordings with about 110,000 expert beat annotations (PhysioNet, open access). Used for beat-level validation and the live replay demo.
Both are public, de-identified datasets, so no patient-consent issues.

Check on the live form: the solution box shows a 0/200 counter. Confirm whether it counts words or characters. If it counts characters, shorten the solution to fit.

7. Architecture (one flow to demo)

ECG stream (replayed MIT-BIH)	Filter + windowing	1D-CNN risk score	Grad-CAM + Granite reason	Dashboard + alert log

8. How we score higher

- **Show the latency number** (ms per window) on a slide. 'Real-time' has to be proven.
- **Lead with explainability.** Clinicians will not trust a black box, and it is our differentiator.
- **Use IBM tools visibly:** watsonx.ai and Granite in the demo and the slides.
- **Report the right metrics:** precision, recall, F1, false-alarm rate. Never plain accuracy.
- **Demo one clean end-to-end flow:** signal arrives, gets scored, reason appears.
- **Say it is decision support, not diagnosis.**

9. Be honest about limits (judges respect this)

- The stream is a **replay** of stored recordings, not live patient data.
- Both datasets are old and international (PTB-XL from Germany, MIT-BIH from the USA). Next step would be validation on local clinical data.
- MIT-BIH has only 47 subjects, so train on PTB-XL and use MIT-BIH for validation and the demo.
- The latency target (for example under 100 ms per window) is a goal. Report whatever we actually measure.

10. Open questions for the team

- Who owns which task (fill the Owner column)?
- What is the submission deadline and is a demo video or PPT also required?
- Does everyone have IBM Cloud / watsonx.ai access, or do we use the fallback for explanations?
- Compute: Colab free tier or Kaggle GPU for training?